

- RAG Learning

4.10 Fundamentals of Databases

Computer Science — AQA A-Level (7517) — Paper 2: Written Theory

Topic 4.10 is where the abstract theory of data structures meets the messy practical reality of storing information that real organisations depend on. A database is not just "a spreadsheet with rules"; it is a carefully engineered system whose design decisions — what entities to model, which attributes to make keys, how to break tables apart to remove redundancy, how to protect the data from concurrent access and from malicious users — ripple through every application built on top of it. Examiners reward candidates who treat database design as a sequence of deliberate choices with justifications: why this key, why this normal form, why this join, why this isolation level. Memorising SQL syntax alone is not enough; you must be able to argue design.

Exam Tip: Many 8–12 mark database questions follow a predictable structure: given a denormalised scenario, identify problems (redundancy, anomalies), normalise to 3NF showing the intermediate 1NF and 2NF tables, draw the ER diagram, and write SQL queries. Practise the full pipeline end-to-end — examiners award method marks at each stage even if a later stage goes wrong.

4.10.1 Conceptual Data Model — Entity Relationship Modelling

Before any tables are created, a database designer builds a **conceptual data model**: a technology-neutral picture of what entities exist in the problem domain, what attributes each has, and how they relate. The tool for this is the **Entity Relationship Diagram (ERD)**.

Entity — a thing of interest in the problem domain about which data will be stored. Entities become tables in the relational implementation. Examples: *Student*, *Course*, *Book*, *Order*.

Attribute — a property or characteristic of an entity. Attributes become columns of the table. Examples: a Student entity has attributes *StudentID*, *Surname*, *Forename*, *DateOfBirth*.

Relationship — an association between two (or occasionally more) entities that captures some real-world fact. Examples: a Student *enrols on* a Course; a Customer *places* an Order.

Cardinality

Every relationship has a **cardinality** (or degree) describing how many instances of one entity can relate to how many instances of the other. The three A-Level cases are:

- **One-to-one (1:1)** — each A relates to at most one B and vice versa. Example: a Person has one Passport; each Passport belongs to one Person.
- **One-to-many (1:M)** — each A relates to many Bs, but each B relates to exactly one A. Example: a Customer places many Orders; each Order belongs to exactly one Customer.
- **Many-to-many (M:M)** — each A relates to many Bs and each B relates to many As. Example: a Student takes many Courses; a Course is taken by many Students.

Notation — Chen vs crow's foot

Two notations dominate. **Chen notation** (1976) draws entities as rectangles, attributes as ellipses hanging off the rectangle, and relationships as a diamond on the line between entities, with cardinality labels (1, M, N) at each end. It is verbose but expressive and common in textbooks.

Crow's foot notation (UK A-Level default) omits the diamond and annotates each end of the line with a symbol: a single bar means "exactly one", a circle means "zero", and a three-pronged "crow's foot" means "many". You read each end separately: a line with a crow's foot at the Course end and a single bar at the Enrolment end reads "one Enrolment relates to many Courses" — except of course you read it the other way for the relationship to make sense.

Worked ERD — Student, Course, Enrolment

A student can enrol on many courses and a course has many students — a classic M:M. Relational databases cannot implement M:M directly because a single foreign key column cannot hold a list.

The resolution is a **link entity** (also called an associative or junction entity) that sits between the two and splits the relationship into two 1:M relationships.

```
STUDENT  ----<  ENROLMENT  >----  COURSE
(1)          (M)      (M)          (1)
```

```
Student(StudentID PK, Surname, Forename, DateOfBirth)
Course (CourseID PK, Title, Level, Credits)
Enrolment(StudentID FK, CourseID FK, EnrolDate, Grade)
-- composite PK = (StudentID, CourseID)
```

The link entity Enrolment has a **composite primary key** made of the two foreign keys to Student and Course. It can also carry its own attributes that describe the relationship itself (the enrolment date, the grade eventually awarded) — facts that belong to neither Student nor Course alone.

Worked ERD — Customer, Order, Product

```
CUSTOMER  ----<  ORDER  ----<  ORDERLINE  >----  PRODUCT
(1)          (M)   (1)   (M)      (M)          (1)
```

```
Customer (CustomerID PK, Name, Email, Address)
Order    (OrderID PK, OrderDate, CustomerID FK)
OrderLine (OrderID FK, ProductID FK, Quantity, PriceAtSale)
Product  (ProductID PK, Description, UnitPrice, StockLevel)
```

A Customer places many Orders (1:M). An Order contains many Products and a Product appears on many Orders, so OrderLine resolves that M:M. Note that PriceAtSale is stored on OrderLine rather than derived from Product.UnitPrice — because prices change over time and a historical invoice must reflect the price charged that day, not today's.

Exam Tip: When you see "many-to-many" in an exam scenario, resolving it with a link entity is nearly always expected. State it explicitly: "The M:M between Student and Course is resolved by a link entity Enrolment, whose composite primary key (StudentID, CourseID) combines the two foreign keys." That sentence alone often scores 3 marks.

4.10.2 Relational Databases

The relational model, invented by E. F. Codd in 1970, is the mathematical foundation of every mainstream SQL database (Oracle, SQL Server, PostgreSQL, MySQL, SQLite).

Relation — a two-dimensional table. Each **tuple** (row) represents one instance of the entity; each **attribute** (column) represents one property. The **domain** of an attribute is the set of permitted values (e.g. the domain of Age might be integers 0–150).

Keys

Primary key — an attribute (or minimal set of attributes) whose value uniquely identifies each tuple in the relation. Every relation must have exactly one. Cannot be NULL (entity integrity).

Foreign key — an attribute in one relation whose values must match the primary key of another (or the same) relation. Enforces referential integrity: you cannot reference a row that does not exist.

Composite key — a primary key made up of two or more attributes because no single attribute is unique on its own. Typically found on link entities: (StudentID, CourseID) together identify an Enrolment, but neither alone does.

- **Candidate key** — any attribute (or minimal set) that *could* serve as primary key. A relation may have several candidate keys; the designer chooses one to be primary.
- **Secondary (alternate) key** — a candidate key not chosen as primary. Often given a UNIQUE index so lookups on it are fast.

Integrity rules

- **Entity integrity.** No component of a primary key may be NULL. If the PK could be NULL you could not distinguish two NULL-keyed rows.

- **Referential integrity.** Every non-NULL foreign key value must match an existing primary key value in the referenced table. This prevents "orphan" rows — an Order referring to a CustomerID that has been deleted.

Exam Tip: Referential integrity does not prevent *bad data*, it prevents *dangling references*. A foreign key still lets you type any valid CustomerID — it just refuses CustomerIDs that do not exist. Students often confuse this with data validation.

4.10.3 Database Design and Normalisation

A poorly designed schema stores the same fact in several places. That causes three classes of problem:

- **Update anomaly.** If a customer's address is stored on every one of their orders, changing the address requires updating every row. Miss one and the data is now inconsistent.
- **Insert anomaly.** You cannot record a new course until at least one student enrolls on it, if course data only lives on enrolment rows.
- **Delete anomaly.** Deleting the last student enrolled on a course also deletes the only copy of that course's information.

Normalisation — a formal, step-by-step process of decomposing relations to eliminate redundancy and the anomalies it causes, while preserving the information content. Each step removes a specific category of bad dependency.

1NF — First Normal Form

1NF — every attribute value is atomic (no lists, sets or nested tables inside a cell), there are no repeating groups, and a primary key exists.

An "Orders" column holding "Milk, Bread, Eggs" violates 1NF: the cell is not atomic. 1NF requires splitting that into multiple rows (or a separate OrderLine table).

2NF — Second Normal Form

2NF — the relation is in 1NF and every non-key attribute depends on the *whole* of the primary key, not just part of it. Only relevant when the primary key is composite; if the PK is a single attribute, 1NF implies 2NF.

3NF — Third Normal Form

3NF — the relation is in 2NF and no non-key attribute transitively depends on the primary key via another non-key attribute. In other words, non-key attributes depend only on the key, the whole key and nothing but the key.

Worked normalisation example

Start with an unnormalised "enrolment sheet":

StudentID	Name	Courses taken
S01	Ali Khan	CS101 Databases (Dr Smith, Room 12); MA201 Calculus (Dr Green, Room 07)
S02	Beth Lee	CS101 Databases (Dr Smith, Room 12)

Step 1 → 1NF. Split the repeating "Courses taken" column into separate atomic rows:

StudentID	Name	CourseID	CourseTitle	Tutor	Room
S01	Ali Khan	CS101	Databases	Dr Smith	12
S01	Ali Khan	MA201	Calculus	Dr Green	07
S02	Beth Lee	CS101	Databases	Dr Smith	12

Composite PK = (StudentID, CourseID). The table is in 1NF.

Step 2 → 2NF. Examine partial dependencies. *Name* depends only on *StudentID* (half of the key). *CourseTitle*, *Tutor*, *Room* depend only on *CourseID* (the other half). These are partial dependencies — bad. Split into three tables:

```
Student  (StudentID, Name)
Course   (CourseID, CourseTitle, Tutor, Room)
Enrolment(StudentID, CourseID)
```

Step 3 → 3NF. Examine the *Course* table. Does any non-key attribute depend on another non-key attribute? *Room* arguably depends on *Tutor* (each tutor has their own office), not directly on *CourseID*. That is a transitive dependency: *CourseID* → *Tutor* → *Room*. Factor out:

```
Student  (StudentID, Name)
Course   (CourseID, CourseTitle, Tutor)      -- Tutor is FK to Staff
Staff    (Tutor, Room)
Enrolment(StudentID, CourseID)
```

Now every non-key attribute in every relation depends on the key, the whole key, and nothing but the key. The schema is in 3NF.

Beyond 3NF

Boyce–Codd Normal Form (BCNF) is a strengthening of 3NF that handles rare cases where a non-trivial dependency's determinant is not a superkey. **4NF** deals with multi-valued dependencies. Neither is core to AQA 7517, but awareness is worth a mark in a "what normal forms exist beyond 3NF" question. In practice most commercial schemas stop at 3NF; chasing higher forms often produces so many tables that joins become a performance problem.

Exam Tip: The memorable rhyme for 3NF: "each non-key attribute depends on the key, the whole key, and nothing but the key" — attributed to Bill Kent. "The key" = 1NF (there is a key), "the whole key" = 2NF (no partial dependency), "nothing but the key" = 3NF (no transitive dependency). Reciting this in an exam answer is worth marks.

4.10.4 SQL — Structured Query Language

SQL is the declarative language for talking to a relational database. "Declarative" means you specify *what* you want, not *how* to get it; the database's query optimiser chooses the execution plan.

SELECT, FROM, WHERE

```
SELECT Forename, Surname, DateOfBirth
FROM Student
WHERE DateOfBirth >= '2006-09-01'
      AND Surname LIKE 'K%'
ORDER BY Surname ASC, Forename ASC;
```

The clauses are logically processed FROM → WHERE → SELECT → ORDER BY. Operators available in WHERE include:

Operator	Meaning	Example
=	equal	Surname = 'Khan'
<> or !=	not equal	Grade <> 'F'
< <= > >=	numeric/date compare	Age >= 18
BETWEEN a AND b	inclusive range	Price BETWEEN 10 AND 20
IN (list)	set membership	Grade IN ('A','B','C')
LIKE '%pattern_'	pattern match: % any sequence, _ any single char	Email LIKE '%@school.ac.uk'
IS NULL	test for NULL (never use = NULL)	MiddleName IS NULL
AND OR NOT	Boolean combinators	Age >= 18 AND City = 'Leeds'

Aggregates, GROUP BY, HAVING

Aggregate functions collapse many rows to one: `COUNT` , `SUM` , `AVG` , `MIN` , `MAX` . Combined with `GROUP BY` they produce per-group summaries; `HAVING` filters the groups themselves (whereas `WHERE` filters rows before grouping).

```
SELECT  CourseID,
        COUNT(*)      AS NumStudents,
        AVG(Grade)     AS MeanGrade
FROM    Enrolment
WHERE    EnrolDate >= '2025-09-01'      -- rows first
GROUP BY CourseID                      -- then group
HAVING   COUNT(*) >= 10                 -- then filter groups
ORDER BY MeanGrade DESC;
```

Expected output shape (illustrative):

CourseID	NumStudents	MeanGrade
CS101	42	68.3
MA201	31	64.1
EN301	18	59.7

INSERT, UPDATE, DELETE

```
INSERT INTO Student (StudentID, Surname, Forename, DateOfBirth)
VALUES ('S03', 'Davies', 'Charlie', '2007-02-14');
```

```
UPDATE Student
SET    Email = 'c.davies@school.ac.uk'
WHERE  StudentID = 'S03';
```

```
DELETE FROM Enrolment
WHERE  StudentID = 'S03' AND CourseID = 'CS101';
```

An `UPDATE` or `DELETE` without a `WHERE` clause touches every row — a common and devastating mistake. Always write the `WHERE` first.

Data Definition Language — CREATE / ALTER / DROP

```
CREATE TABLE Student (  
    StudentID    CHAR(4)        PRIMARY KEY,  
    Surname      VARCHAR(40)    NOT NULL,  
    Forename     VARCHAR(40)    NOT NULL,  
    DateOfBirth  DATE           NOT NULL,  
    Email        VARCHAR(100)    UNIQUE  
);  
  
CREATE TABLE Course (  
    CourseID     CHAR(5)        PRIMARY KEY,  
    Title        VARCHAR(80)    NOT NULL,  
    Credits      INT            CHECK (Credits BETWEEN 1 AND 60)  
);  
  
CREATE TABLE Enrolment (  
    StudentID    CHAR(4),  
    CourseID     CHAR(5),  
    EnrolDate    DATE           DEFAULT CURRENT_DATE,  
    Grade        CHAR(1),  
    PRIMARY KEY (StudentID, CourseID),  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);  
  
ALTER TABLE Student ADD COLUMN TutorGroup VARCHAR(5);  
DROP TABLE Enrolment;      -- deletes both data and schema
```

4.10.5 SQL Joins

A join combines rows from two (or more) tables on a matching condition, usually foreign-key-to-primary-key.

Join type	Rows returned	Typical use
INNER JOIN	only rows with matches in both tables	"orders with customer info"

LEFT (OUTER) JOIN	all rows from left + matched rows from right; NULL if no match	"all customers, with their orders if any"
RIGHT (OUTER) JOIN	mirror of LEFT	rare in practice; rewrite as LEFT JOIN with tables reversed
FULL OUTER JOIN	all rows from both sides; NULLs where unmatched	reconciliation / diff between two sources
Self-join	joins a table to itself via aliases	"find each employee's manager" from an Employee(ID, ManagerID) table

Example data — Customer:

CustomerID	Name
C1	Ali
C2	Beth
C3	Chen

Order:

OrderID	CustomerID	Total
O100	C1	42.00
O101	C1	15.50
O102	C2	99.00

```
SELECT c.Name, o.OrderID, o.Total
FROM   Customer c
INNER JOIN Order o ON o.CustomerID = c.CustomerID;
```

INNER JOIN result (Chen has no orders, so is excluded):

Name	OrderID	Total
Ali	O100	42.00
Ali	O101	15.50
Beth	O102	99.00

```
SELECT c.Name, o.OrderID, o.Total
FROM   Customer c
LEFT JOIN Order o ON o.CustomerID = c.CustomerID;
```

LEFT JOIN result (Chen included with NULLs):

Name	OrderID	Total
Ali	O100	42.00
Ali	O101	15.50
Beth	O102	99.00
Chen	NULL	NULL

Exam Tip: When asked "what is the difference between INNER and LEFT JOIN?" always answer with a concrete example of a row that would be *dropped* by INNER but *kept* (with NULLs) by LEFT. Abstract definitions alone rarely earn full marks.

4.10.6 Client-Server Databases

Small applications use an embedded database: the program links against a library (SQLite, Access) and reads/writes a file on local disk. This breaks once multiple users need simultaneous access, or when the data must survive the client machine.

A **client-server DBMS** (Oracle, SQL Server, PostgreSQL, MySQL) runs as a long-lived server process on a dedicated machine. Clients — web servers, mobile apps, reporting tools — open network connections and send SQL strings; the server parses, optimises, executes and returns result sets. Key consequences:

- **Concurrent access.** Hundreds or thousands of clients can query and modify simultaneously. The DBMS provides locking and transaction isolation (see 4.10.9–10) to prevent them corrupting each other's work.
- **Centralised integrity.** Constraints (PK, FK, CHECK) are enforced in one place; no application can bypass them by writing directly to the files.
- **Separation of concerns.** Hardware, backups, security, tuning are administered on the server; clients see a uniform SQL interface regardless of their OS or language.
- **Scalability.** Vertical scaling (bigger server) is straightforward; horizontal scaling (many servers) is hard for relational systems because transactions must remain consistent — discussed further under Big Data.

4.10.7 Big Data

By the mid-2000s web-scale systems (Google, Amazon, Facebook) were generating datasets too large, too fast and too varied for traditional relational databases on single servers. The label **Big Data** captures that shift.

The five Vs

- **Volume** — terabytes and petabytes, not gigabytes. Facebook ingests multiple petabytes per day.
- **Velocity** — data arrives continuously and often must be processed in near real-time (stock tickers, sensor streams, tweets).
- **Variety** — structured rows are only a fraction; the rest is JSON, XML, images, video, log lines, sensor readings.
- **Veracity** — data is noisy, incomplete and sometimes wrong; cleaning is a major cost.
- **Value** — the payoff: analytics, recommendations, fraud detection, research insights.

Why relational databases struggle

Relational systems give ACID guarantees by serialising writes through a central authority. That works beautifully up to perhaps one very large server — but horizontal scaling across many machines (a shared-nothing cluster) forces a choice: either give up full consistency (see the *CAP theorem*: among Consistency, Availability and Partition-tolerance you can have any two) or accept the huge coordination cost of distributed transactions. Web-scale systems almost always relax consistency in favour of availability and partition-tolerance.

NoSQL families

Type	Example	Data model	When to use
Document	MongoDB, CouchDB	JSON-like documents grouped in collections	flexible, evolving schemas — user profiles, CMS content
Key-value	Redis, DynamoDB	simple key → value lookup	caches, session stores, feature flags; microsecond latency
Column-family	Cassandra, HBase	sparse wide rows grouped into column families	write-heavy time-series, event logging at petabyte scale
Graph	Neo4j, Neptune	nodes and edges with properties	social graphs, knowledge graphs, fraud rings, recommendation

NoSQL — "Not Only SQL". A family of non-relational databases designed for horizontal scalability, flexible schemas and high availability. They typically relax ACID to *BASE* (Basically Available, Soft-state, Eventual consistency).

Distributed storage — HDFS

The **Hadoop Distributed File System (HDFS)** stores enormous files by chopping them into 128 MB blocks and spreading the blocks across a cluster of commodity machines. Each block is replicated (default three copies) on different nodes so that any single-machine failure loses nothing. A single *NameNode* holds the directory tree and block-location metadata; many

DataNodes hold the blocks themselves. This is the substrate on top of which MapReduce and its successors run.

MapReduce

MapReduce — a programming model for processing very large datasets across a cluster. The computation is expressed as two functions: *map* (independent transformation of each input record into key-value pairs) and *reduce* (aggregation of all values sharing a key). The framework handles splitting, distributing, shuffling, fault-tolerance and aggregation.

```
# Word count: how many times does each word appear across a corpus?

function map(docID, docText):
    for word in tokenise(docText):
        emit(word, 1)

function reduce(word, listOfOnes):
    emit(word, sum(listOfOnes))

# Input docs:    "the cat sat" , "the dog sat"
# map emits:     (the,1)(cat,1)(sat,1)(the,1)(dog,1)(sat,1)
# shuffle:       the -> [1,1] ; cat -> [1] ; sat -> [1,1] ; dog -> [1]
# reduce emits:  (the,2)(cat,1)(sat,2)(dog,1)
```

The power is that the same code runs over 1 GB on a laptop or 10 PB on 10,000 machines — the framework handles the scaling.

4.10.8 Functional Programming and Databases

There is a deep kinship between the relational model and the functional paradigm. A relation is an immutable set of tuples; SQL's SELECT is a pure function producing a new relation from existing ones without mutating input data. This is why SQL is called *declarative*: you describe the transformation, not the stepwise mutation.

In Python, a list comprehension mirrors relational algebra. Selection (WHERE) corresponds to the *if* clause; projection (SELECT list) corresponds to the expression before *for*:

```
# SQL:  SELECT Forename FROM Student WHERE DateOfBirth >= '2006-09-01';
# Python equivalent:
forenames = [s.forename for s in students if s.dob >= date(2006,9,1)]
```

Immutability makes reasoning about distributed computation (MapReduce, Spark) dramatically easier: there are no shared mutable structures to lock, so the same logic parallelises across thousands of machines without race conditions.

4.10.9 Transactions and ACID

Transaction — a unit of work comprising one or more SQL statements that must succeed or fail as an atomic whole. Bracketed by `BEGIN TRANSACTION` and either `COMMIT` (make changes permanent) or `ROLLBACK` (undo all changes since `BEGIN`).

ACID — the four guarantees a transactional DBMS provides: **A**tomicity, **C**onsistency, **I**solation, **D**urability.

Property	Meaning	How it is enforced
Atomicity	all statements commit or none do — no partial effect	undo log / rollback segments; on crash, incomplete transactions are rolled back during recovery
Consistency	all declared constraints (PK, FK, CHECK, triggers) hold before and after the transaction	constraint checking at commit time; transaction aborts if any constraint would be violated
Isolation	concurrent transactions produce the same result as if they had run one at a time in some order (serialisable)	locking (2PL) or multiversion concurrency control (MVCC); tunable by isolation level
Durability	once <code>COMMIT</code> returns, the change survives crashes, power loss, disk	write-ahead log (WAL) flushed to non-volatile storage before <code>COMMIT</code>

failures

reports success; replication

Worked example — bank transfer

```

BEGIN TRANSACTION;

UPDATE Account
SET    Balance = Balance - 100.00
WHERE  AccountID = 'A123';

IF (SELECT Balance FROM Account WHERE AccountID = 'A123') < 0 THEN
    ROLLBACK;           -- insufficient funds; undo the debit
    RAISE 'Insufficient funds';
END IF;

UPDATE Account
SET    Balance = Balance + 100.00
WHERE  AccountID = 'B456';

COMMIT;                -- both updates now permanent

```

Without atomicity a crash between the two updates could debit A123 without crediting B456 — money vanishes. Without isolation, a concurrent reader might observe the interim state where A has been debited but B not yet credited — the books don't balance. Without durability, a crash immediately after COMMIT might lose the transaction even though the customer has been told it succeeded.

Isolation levels

Level	Prevents	Still allows	Cost
READ UNCOMMITTED	nothing	dirty reads, non-repeatable reads, phantoms	cheapest; almost never used
READ COMMITTED	dirty reads	non-repeatable reads, phantoms	default in PostgreSQL / SQL Server

REPEATABLE READ	dirty + non-repeatable reads	phantom rows (new rows appearing)	default in MySQL InnoDB
SERIALIZABLE	everything — transactions appear fully serial	nothing	highest overhead; deadlocks/aborts more frequent

Higher isolation gives stronger guarantees at the cost of concurrency (more locks, more waiting, more aborts). A banking system must use SERIALIZABLE or equivalent; a news feed can get away with READ COMMITTED.

Exam Tip: Memorise ACID by imagining each letter failing: A fails = money teleports, C fails = constraints lie, I fails = other users see half-done work, D fails = commits evaporate after a power cut. Linking each letter to a concrete disaster cements the meaning.

4.10.10 Locking and Concurrency Control

Isolation in a relational DBMS is typically implemented by **locks**. Before a transaction reads a row it acquires a *shared lock* (multiple readers allowed); before it writes it acquires an *exclusive lock* (one writer, no concurrent readers on the same row).

Two-phase locking (2PL)

2PL guarantees serialisability by requiring every transaction to proceed in two phases: a *growing phase* in which locks are acquired but never released, followed by a *shrinking phase* in which locks are released but never acquired. In practice most DBMSs use *strict 2PL*: all locks are held until COMMIT or ROLLBACK, which also prevents cascading aborts.

Deadlock

A **deadlock** occurs when two or more transactions are each waiting for locks held by the other — a cycle in the wait-for graph. Classic pattern:

Transaction T1:	Transaction T2:
LOCK row A (granted)	LOCK row B (granted)
LOCK row B (waits on T2)	LOCK row A (waits on T1)

Neither can progress. Responses:

- **Detection.** The DBMS periodically examines the wait-for graph for cycles; on finding one it aborts a chosen "victim" transaction (usually the youngest or the one with fewest locks) and the other proceeds. The victim retries.
- **Prevention.** Impose a global ordering on resources and require every transaction to acquire locks in that order (no transaction can ever be waiting "behind" another in a cycle).
- **Timeouts.** A simple pragmatic fallback: if a lock wait exceeds a threshold, abort.

Timestamp ordering

An alternative to locking: each transaction is stamped with a start time; conflicting operations are resolved by aborting the transaction whose timestamp violates the serial order. No locks are taken, so no deadlocks — but more transactions may abort and retry. MVCC (multiversion concurrency control, used by PostgreSQL, Oracle) combines timestamping with multiple stored row versions so readers never block writers and vice versa.

4.10.11 Data Integrity Constraints

Constraints are declarations on a table that the DBMS enforces automatically on every INSERT or UPDATE. They are the first and cheapest line of data-quality defence.

Constraint	Guarantee	Example
NOT NULL	value must be supplied	Surname VARCHAR(40) NOT NULL
UNIQUE	no two rows share this value (NULLs usually allowed)	Email VARCHAR(100) UNIQUE
PRIMARY KEY	NOT NULL + UNIQUE, and marks the table's identifier	StudentID CHAR(4) PRIMARY KEY

FOREIGN KEY	value must exist as PK in referenced table	FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)
CHECK	arbitrary Boolean predicate must hold	CHECK (Credits BETWEEN 1 AND 60)
DEFAULT	value used when none supplied	EnrolDate DATE DEFAULT CURRENT_DATE

```
CREATE TABLE Employee (
    EmpID    CHAR(6)      PRIMARY KEY,
    Email    VARCHAR(100) UNIQUE NOT NULL,
    DeptID   CHAR(3)      NOT NULL,
    Salary   DECIMAL(9,2) CHECK (Salary >= 0),
    Hired    DATE         DEFAULT CURRENT_DATE,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

`ON DELETE` and `ON UPDATE` clauses on a foreign key specify what happens when the referenced row is deleted or its key changes: `CASCADE` propagates the action, `RESTRICT` refuses the delete, `SET NULL` nulls the FK, `SET DEFAULT` resets it.

4.10.12 Database Security

Access rights — GRANT and REVOKE

A DBMS authenticates users and authorises operations by role. `GRANT` adds a privilege; `REVOKE` removes it.

```
-- Grant read-only access on Student to the reporting role
GRANT SELECT ON Student TO reporting_role;

-- Grant full CRUD on Enrolment to the registrar role
GRANT SELECT, INSERT, UPDATE, DELETE ON Enrolment TO registrar_role;
```

```
-- Remove delete permission when retention policy tightens
REVOKE DELETE ON Enrolment FROM registrar_role;
```

SQL injection

SQL injection — an attack in which user-supplied input is concatenated into an SQL string, allowing the attacker to terminate the intended query and append their own SQL. Defeated by *parameterised queries* (prepared statements) which send the SQL template and parameters separately, so user input can never be parsed as SQL syntax.

Vulnerable code	Parameterised fix
<pre>"SELECT * FROM User WHERE name='" + n + "'" AND pw='" + p + "'"</pre>	<pre>"SELECT * FROM User WHERE name=? AND pw=?" with (n, p) bound</pre>

```
# BAD -- vulnerable, user can inject SQL
name = request.form['username']          # attacker sends: ' OR '1'='1' --
pw   = request.form['password']
sql  = "SELECT * FROM User WHERE name='" + name + "'" AND pw='" + pw + "'"
# resulting query is:  SELECT * FROM User WHERE name=' ' OR '1'='1' --' AND pw='
# which is true for every row -- login bypass.
cursor.execute(sql)

# GOOD -- parameterised; user input is data, never code
cursor.execute(
    "SELECT * FROM User WHERE name = ? AND pw = ?",
    (name, pw)
)
```

Other security measures:

- **Auditing.** Every DDL and security-sensitive DML is logged with user, timestamp, old/new values. Allows forensic reconstruction after a breach.
- **Encryption at rest.** The physical data files (and backups) are encrypted so that theft of the disk or tape does not expose the contents. Typically AES-256 with keys held in a separate key-management system.

- **Encryption in transit.** TLS on the client–server connection prevents network eavesdropping.
- **Backups.** Regular full + incremental backups stored off-site; routinely tested by restoring to a scratch server. "A backup you have never restored from is not a backup."
- **Principle of least privilege.** Each application connects with a role that has only the permissions it actually needs — never as the DBA.

Case Study: Knight Capital, 1 August 2012. The US high-frequency trading firm Knight Capital deployed new trading software to eight production servers. A critical configuration flag was set correctly on seven servers but not on the eighth, where legacy code was reactivated. In 45 minutes the misconfigured server sent millions of erroneous orders, accumulating a \$460 million loss — more than four times the firm's annual profit. The company was effectively bankrupted within a day. The incident is a textbook example of how a database/deployment failure is rarely "just a bug": the weaknesses were in change-management (no atomic deployment across the cluster), auditing (no real-time check that all servers were on the same version), and rollback (no kill-switch to halt trading when the anomaly was detected). Every ACID property matters at scale, but so does the surrounding operational discipline.

Case Study: Facebook's MySQL + memcached architecture. Facebook famously ran (and largely still runs) user data on thousands of sharded MySQL instances fronted by a vast memcached layer. Rather than move to a NoSQL store, Facebook invested heavily in making relational storage scale horizontally: user IDs are sharded across servers, reads are served from memcached caches with eventual consistency, and writes go to a single authoritative shard. The lesson for exam answers: "big data" does not automatically mean "NoSQL". The choice depends on access patterns, consistency requirements and operational expertise. Relational with caching is often faster and more reliable than a naive migration to a document store.

Worked exam-style question — full pipeline

A worked walk-through shows how the topic's ideas combine. Scenario: a small library wants to track books, members and loans. A member can borrow many books (one copy at a time per title),

and a book can be borrowed over its lifetime by many members. The librarian initially proposes a single spreadsheet:

MemberID	MemberName	Phone	Books borrowed
M01	Ali Khan	07700 900 111	B010 Pride & Prejudice (out 02-Mar, back 09-Mar); B041 Dune (out 05-Mar)
M02	Beth Lee	07700 900 222	B010 Pride & Prejudice (out 11-Mar)

Problems. The "Books borrowed" cell is not atomic (1NF violation). Member details repeat on every loan (update anomaly: changing a phone number requires editing many rows). You cannot record a newly-acquired book until someone borrows it (insert anomaly). Deleting the last loan of a book loses the book (delete anomaly).

Normalised design (3NF).

```
Member(MemberID, Name, Phone, JoinDate)
Book (BookID, Title, Author, ISBN, Acquired)
Loan (LoanID, MemberID FK, BookID FK, DateOut, DateBack)
```

Loan uses a surrogate LoanID rather than (MemberID, BookID) as PK because a member may borrow the same title again later — the historical loans must all be distinguishable. MemberID and BookID become foreign keys.

ERD (crow's foot).

```
MEMBER  ---| |----<  LOAN  >----| |---  BOOK
(1)          (M)    (M)          (1)
```

SQL — schema.

```
CREATE TABLE Member (
    MemberID CHAR(4)      PRIMARY KEY,
    Name      VARCHAR(80) NOT NULL,
    Phone     VARCHAR(20),
    JoinDate  DATE        DEFAULT CURRENT_DATE
```

```
);

CREATE TABLE Book (
    BookID    CHAR(5)        PRIMARY KEY,
    Title     VARCHAR(200) NOT NULL,
    Author    VARCHAR(100),
    ISBN      CHAR(13)       UNIQUE,
    Acquired  DATE
);

CREATE TABLE Loan (
    LoanID    INT            PRIMARY KEY,
    MemberID  CHAR(4)        NOT NULL,
    BookID    CHAR(5)        NOT NULL,
    DateOut   DATE           NOT NULL,
    DateBack  DATE,          -- NULL while still on loan
    CHECK (DateBack IS NULL OR DateBack >= DateOut),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (BookID)   REFERENCES Book(BookID)
);
```

SQL — typical queries.

```
-- Q1: list members with books currently out
SELECT m.Name, b.Title, l.DateOut
FROM   Loan l
INNER JOIN Member m ON m.MemberID = l.MemberID
INNER JOIN Book b ON b.BookID = l.BookID
WHERE  l.DateBack IS NULL
ORDER BY l.DateOut;

-- Q2: top 5 most-borrowed books of the year
SELECT b.Title, COUNT(*) AS LoanCount
FROM   Loan l
JOIN   Book b ON b.BookID = l.BookID
WHERE  l.DateOut BETWEEN '2026-01-01' AND '2026-12-31'
GROUP BY b.Title
HAVING COUNT(*) >= 3
ORDER BY LoanCount DESC
LIMIT 5;

-- Q3: members who have never borrowed anything (candidates for a reminder email
SELECT m.Name, m.Phone
```

```
FROM    Member m
LEFT    JOIN Loan l ON l.MemberID = m.MemberID
WHERE   l.LoanID IS NULL;
```

Query 3 is a classic LEFT JOIN + IS NULL idiom for "rows in the left table with no matching right-table rows" — useful whenever you need to find absences rather than presences.

Exam Tip: The "LEFT JOIN ... WHERE ... IS NULL" pattern is commonly tested. If a question asks "find all X that have no Y", resist the temptation to use a subquery (NOT IN / NOT EXISTS are alternatives, but NOT IN behaves surprisingly around NULLs). The LEFT JOIN pattern is fastest on most optimisers and the easiest to explain in writing.

Putting it together — a checklist for design questions

1. Identify entities (nouns in the scenario).
2. List attributes for each entity; underline the primary key.
3. Identify relationships (verbs) and their cardinality.
4. Resolve any M:M with a link entity whose PK is composite.
5. Check 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency).
6. Declare constraints: PK, FK, NOT NULL, UNIQUE, CHECK.
7. Write the SQL: CREATE TABLEs first, then queries; use joins to recombine what normalisation split apart.
8. Consider concurrency (transactions, isolation level) and security (roles, parameterised queries, encryption).

The examiner's ideal script reads like a design diary: each decision is named and justified, each SQL fragment annotated with its purpose. Database questions are rarely won on raw syntax — they are won on the coherence of the design narrative.